```
import heapq
initial = ( 1,2,3,
          4, 0, 6,
          7, 5, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def misplaced_tiles(state, goal):
    count = 0

    for i in range(9):
        if state[i] != 0 and state[i] != goal[i]:
            count += 1

    return count

def manhattan_distance(state, goal):
    distance = 0

    for tile in range(1, 9):

        current_position = state.index(tile)
        goal_position = goal.index(tile)

        current_row = current_position // 3
        current_col = current_position % 3

        goal_row = goal_position // 3
        goal_col = goal_position % 3

        distance += abs(current_row - goal_row)
        distance += abs(current_col - goal_col)

    return distance

def get_neighbors(state):

    neighbors = []

    zero_position = state.index(0)

    row = zero_position // 3
```

```
    col = zero_position % 3

    moves = [
        (-1, 0),   # UP
        (1, 0),    # DOWN
        (0, -1),   # LEFT
        (0, 1)     # RIGHT
    ]

    for row_change, col_change in moves:

        new_row = row + row_change
        new_col = col + col_change

        if 0 <= new_row < 3 and 0 <= new_col < 3:

            new_position = new_row * 3 + new_col

            child = list(state)

            # Swap blank with tile
            child[zero_position], child[new_position] = \
                child[new_position], child[zero_position]

            neighbors.append(tuple(child))

    return neighbors

def a_star(initial, goal, heuristic):

    # Priority queue
    # (f, g, state, path)
    open_list = []

    g = 0
    h = heuristic(initial, goal)
    f = g + h

    heapq.heappush(open_list, (f, g, initial, [initial]))

    closed = set()

    while open_list:
```

```
        f, g, current, path = heapq.heappop(open_list)

        # Goal test
        if current == goal:
            return path
        if current in closed:
            continue

        closed.add(current)
        for child in get_neighbors(current):

            if child in closed:
                continue

            child_g = g + 1
            child_h = heuristic(child, goal)
            child_f = child_g + child_h

            child_path = path + [child]

            heapq.heappush(
                open_list,
                (child_f, child_g, child, child_path)
            )

    return None
def print_solution(path):

    for step, state in enumerate(path):

        print("\nStep", step)

        for i in range(0, 9, 3):
            print(state[i], state[i + 1], state[i + 2])

    print("\nTotal moves:", len(path) - 1)


print("====================================")
print("A* USING MISPLACED TILES")
print("====================================")

path1 = a_star(initial, goal, misplaced_tiles)
```

```
print_solution(path1)


print("\n\n=====================================")
print("A* USING MANHATTAN DISTANCE")
print("=====================================")

path2 = a_star(initial, goal, manhattan_distance)

print_solution(path2)
```

OUTPUT:

```
==================================
A* USING MISPLACED TILES
==================================

Step 0
1 2 3
4 0 6
7 5 8

Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Total moves: 2


=====================================
A* USING MANHATTAN DISTANCE
=====================================

Step 0
1 2 3
4 0 6
7 5 8

Step 1
```

```
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Total moves: 2
```